

[테크살롱] 이벤트 소싱과 CQRS

🔗 URL	https://www.youtube.com/watch?v=fA0wkkxsE4U&t=123s
☰ 구분	테크살롱
📅 날짜	2026년 4월 27일
🕒 레벨	레벨1
🔗 블로그글	https://junsangcho.tistory.com/25
⚙ 상태	진행 중

이벤트 소싱이란?

상태가 아닌 이벤트(사건)를 저장하는 방식

- 이벤트 소싱이란 무엇일까? 어플리케이션의 모든 상태 변화를 순서에 따라 이벤트로 보관하는 저장 패턴이다.
- 현재 상태 값을 저장/수정/삭제해오던 전통적인 데이터 저장 방식과 다르게, 순차적으로 발생한 이벤트들을 모두 저장하는 방식을 의미한다.

사건을 저장함으로써 얻을 수 있는 이점

그렇다면 전통적인 방식의 상태 값이 아닌 사건(이벤트)을 저장함으로써 얻을 수 있는 이점은 무엇일까?

1. 과거의 이력 정보를 자연스럽게 관리할 수 있다.

- 전통적인 데이터 저장 방식은 현재의 상태만을 저장한다. 따라서 특정 데이터가 어떤 과정을 거쳐 현재 상태에 도달했는지 추적하기 어렵다. 따라서 과거의 이력 정보가 필요한 경우, 별도의 테이블을 추가로 설계해야 한다.
- 하지만 비즈니스가 확장될수록 이력 관리 구조는 점점 더 복잡해진다. 기존 상태를 조회하고, 변경 전 값을 복사해 저장하며, 여러 테이블 사이의 정합성을 맞추기 위한 추가 로직까지 필요해지기 때문이다.

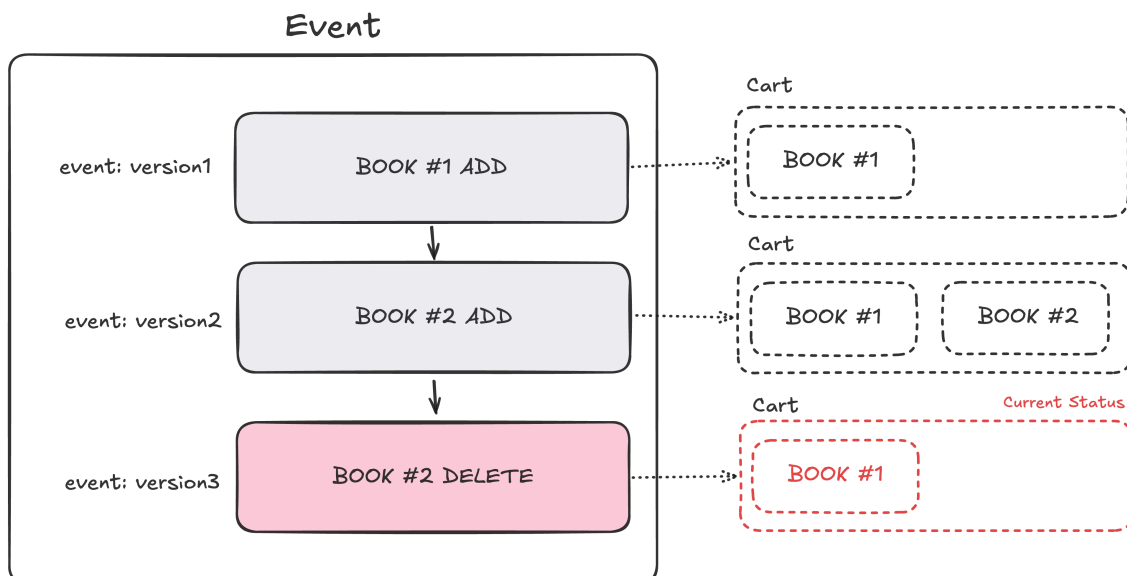
- 반면에 이벤트 소싱은 상태 자체가 아니라 무슨 사건이 발생했는가만을 독립적으로 저장하기 때문에, 오로지 Append 방식으로 데이터 변경 과정을 기록한다. 즉, update가 아닌 새로운 이벤트 추가 작업만 일어나기 때문에 복잡한 이력 관리 로직 없이. 시스템의 모든 변화 과정을 쉽게 추적할 수 있다.
- 특히, 이벤트 로그만으로도 특정 시점의 상태를 재구성하기 쉽기 때문에 디버깅 시점이나, 장애 분석시에도 이점을 갖는다.

2. 이벤트의 Append 방식 덕분에 쓰기 성능이 올라간다.

- 앞서 말했듯이, 각 이벤트는 불변하고 한번 저장된 이후에는 수정(Update, Delete)되지 않는다. 즉, 기존 데이터를 건들이지 않고 단순 추가(Insert)하는 작업만 수행하게 된다.
- 이러한 Append-Only 방식 덕분에 데이터베이스 입장에서는 기존 데이터를 탐색하고 수정,삭제하는 과정이 줄어들기 때문에, 쓰기 경합이나 동시성 처리 부담이 줄어든다. 즉, 오직 Insert만 수행하기 때문에 쓰기 성능이 올라간다.
- 물론, 다른 시스템 요소를 고려한다면, 쓰기 성능이 항상 올라간다고 보장할 수는 없다. 예를 들어, 이벤트 저장 시점에 스냅샷도 저장되고, Projection 갱신과 같은 작업이 함께 수행된다면, 전체 쓰기 작업이 더 복잡해 질 수 있다.
- 따라서 전체 시스템을 고려하지 않은 채 오직 Append 작업만 보았을 때, 얻을 수 있는 이점으로 해석하는 것이 올바르다.

이벤트 소싱 도입 예시

도서 구매 어플리케이션을 예로 이벤트 소싱에 대해 감을 잡아보자. 아래 예시는 사용자 A가 1번 책과 2번 책을 순차적으로 장바구니에 추가하고, 2번 책을 삭제한 상황이다.



- 기존의 결과 값만 저장하던 방식은 장바구니에 1번 책만 담긴 상태로 DB에 저장되어 있겠지만(사진의 Current Status), 이벤트 소싱을 적용한 경우 **1번 추가**, **2번 추가**, **2번 삭제** 라는 3가지 이벤트가 DB에 저장되어 있다. 즉, 이벤트 소싱에서는 Delete와 Update의 개념이 없다. Delete라는 이벤트가 Append 된 것일 뿐이다.

현재 상태는 어떻게 불러오는가? (Event Replay)

- 이벤트 소싱은 현재 상태 값을 그대로 저장하는 것이 아닌, 이벤트를 저장하는 방식이다. 따라서 **현재 상태 값이 필요하다면 DB에 저장된 이벤트들을 기반으로 메모리에서 재구성해야 한다.**
- 이를 Event Replay라고 한다. 이벤트를 모두 조회하고 객체의 현상태를 복원해야 한다. 최신 상태를 알기 위해서는 매번 replay를 해야한다.

스냅 샷이 필요한 이유

- Event Replay의 치명적 단점은 무엇일까? 사용자가 같은 책 A를 10,000,000번 추가하고 삭제하기를 반복했다고 가정하자. 정작 장바구니의 현상태 값은 비어있는데, 이를 재구성하기 위해 이벤트를 10000000번 재생해야 한다. 결국 현재 객체 상태를 불러오기까지의 시간에 트레이드 오프가 발생한다.
- 이를 보완하기 위해 스냅샷 개념을 도입할 수 있다. 스냅샷이란 특정 시점의 상태 값을 별도로 저장함을 의미한다. 만약 비즈니스 정책상 "이벤트 1000개 단위"로 스냅샷을 저장하기로 약속했다면, 1001번째 상태 값이 필요한 경우, 1000번 스냅샷 위에서 1개의 이벤트만 재생하면 된다.
- 스냅 샷은 언제 저장될까? 그리고 어디에 저장할까? 스냅 샷은 이벤트 저장 시점에 생성한다. 하지만 매 이벤트 생성 시점이 아닌, 비즈니스 정책에 맞게 특정 시간 혹은 이벤트 발생 개수를 충족한 경우, 스냅샷을 저장한다.
- 스냅 샷은 일반적으로 Event Store와 분리된 저장소에 저장해야 하는데 성능상 주로 in memory 저장소를 사용한다. 스냅 샷 저장은 일반적으로 캐시의 성향이 짙다.

스냅 샷을 인메모리에 저장하는 이유

- 스냅 샷은 Aggregate를 재구성할 때마다 반복적으로 참조되는 데이터이다. 특히 비슷한 시점의 상태를 연속적으로 재구성하는 경우 동일한 스냅 샷이 여러 번 참조된다.
- 예를 들어 1001번째 상태를 재구성할 때와 1002번째 상태를 재구성할 때를 생각해 보자. 두 경우 모두 1000번째 스냅샷을 읽고, 그 이후 이벤트만 재생하게 된다. 즉, 동일한 스냅샷이 짧은 시간 동안 반복적으로 조회되는 것이다.

- 따라서 스냅 샷은 접근 빈도가 높은 핫 데이터의 성격을 갖기 때문에, 디스크보다 접근 속도가 빠른 인메모리에 저장하는 것이 성능 측면에서 유리하다.

자 여기까지 정리해보자. 이벤트 소싱은 결과 값이 아닌 모든 상태 변화의 이벤트를 저장하는 방식이다. 그리고 이는 수정과 삭제가 아닌 오로지 추가의 방식(append)으로만 데이터가 관리된다. 그렇기에 현재의 도메인 상태를 불러오기 위해서는 이벤트를 불러와 replay 해야 하는데 재생 처리 성능을 위해 스냅샷을 일반적으로 도입한다.

읽기 시점(Query)에도 재구성이 필요할까?

이벤트 소싱 방식을 도입한다면, 쓰기 시점에는 정합성을 보장하기 위해 재구성이 반드시 필요하겠지만, 읽기 시점에도 매순간 재구성을 해줄 필요가 있을까?

만약 10명의 사용자가 동시에 각각 100,000,000번의 이벤트가 수행된 본인 계좌의 현재 상태 정보를 조회한다고 가정해보자. 단순히 10명의 사용자의 현재 계좌 상태를 조회하는 읽기 작업을 수행했을 뿐인데, $10 \times 100,000,000$ 번의 작업이 수반된다. 즉, 이벤트가 증가할수록 성능 부담이 커질 수 있다. 특히 조회 빈도가 높은 시스템에서는 동일한 상태를 반복해서 재구성하는 비용이 부담이 될 수 있다.

이런 이유로, 이벤트 소싱에서는 읽기 성능을 개선하기 위해, CQRS 기반의 Read Model을 별도 구성하는 방안을 일반적으로 도입하는데 이를 알아보자.

이벤트 소싱 도입시에 CQRS패턴을 시스템에 적용하면, **쓰기 영역에서는 이벤트를 기준으로 정합성을 유지하되, 읽기 영역에서는 재구성 비용을 줄이기 위해 이미 계산된 상태를 쉽게 제공할 수 있다.**

CQRS란?

CQRS 패턴이란 무엇일까? 쉽게 말해 시스템 단위에서 읽기/쓰기 요청의 모델을 물리적으로 분리한 패턴을 의미한다.

CQS와 CQRS란?

CQS와 CQRS는 모두 Command와 Query의 책임을 분리하는 것을 의미한다. 차이점은 CQS는 객체,메서드와 같은 코드 단위의 분리를 의미하고, CQRS는 아키텍처 패턴이나 시스템 레벨의 더 넓은 범위에서의 분리를 의미한다.

왜 Command와 Query를 분리하는 것이 중요할까? 코드 단위(CQS)에서 이 중요성을 먼저 이해하자.

아래는 **Command(명령)**와 **Query(조회)**가 섞여 있는 `isSatisfied()` 메서드이다.

```
private final int currentSize;

public int isSatisfied(int threshold) {
    if (currentRate < threshold) {
        resize()           // 상태 변경 (Command)
        return false;      // 값 반환 (Query)
    }
    return true;
}

private void resize(){
    this.currentSize += 5;
}
```

`isSatisfied()` 메서드는 인자 값인 `threshold` 을 기준으로, 현재 상태인 `currentSize` 가 조건을 만족하는지 판단하는 조회 메서드이다. 이때, 조건을 만족하지 않는 경우, `currentSize` 를 재조정하는 상태 변경 메서드(`resize()`)도 함께 실행된다.

이처럼 단일 메서드에서 조회와 상태 변경 작업이 함께 수행된다면, `isSatisfied`를 호출하는 클라이언트는 메서드 호출 시점에 따라 다른 결과 값을 받을 수 도 있다.

```
block.isSatisfied(10) // false 반환
block.isSatisfied(10) // false 반환
...
block.isSatisfied(10) // true 반환
```

즉, 상태 변경인 명령과 조회 개념인 쿼리가 한 메서드 내에 혼용되었기 때문에 발생한 문제이며, 이를 독립적인 메서드로 분리하는 것(CQS)이 필요하다.

CQS를 사용하면 무엇이 좋을까? 우선 “반환 값이 존재하는 메서드”는 다른 개발자에게 쿼리 메서드로 인지되어, Side Effect에 대한 부담이 없다는 의미를 전달할 수 있다. 반면, 반환 값이 없는 void 메서드는 모두 명령으로 해석되어 내부 상태가 변경된다는 주의점을 전달할 수 있다.

이를 시스템 단위로 확장하여, 명령과 쿼리를 **시스템 단위에서 분리한 것**을 CQRS라고 한다. 쉽게 말해, 읽기 요청이 들어오는 Read 모델과 쓰기 요청이 들어오는 Write 모델을 물리적으로 분리한 것이다.

왜 CQRS가 이벤트 소싱과 궁합이 좋을까?

다시 이벤트 소싱으로 돌아오자. 이벤트 소싱의 가장 큰 문제점은 조회 시점에도 발생하는 불필요한 재구성 비용이었다.

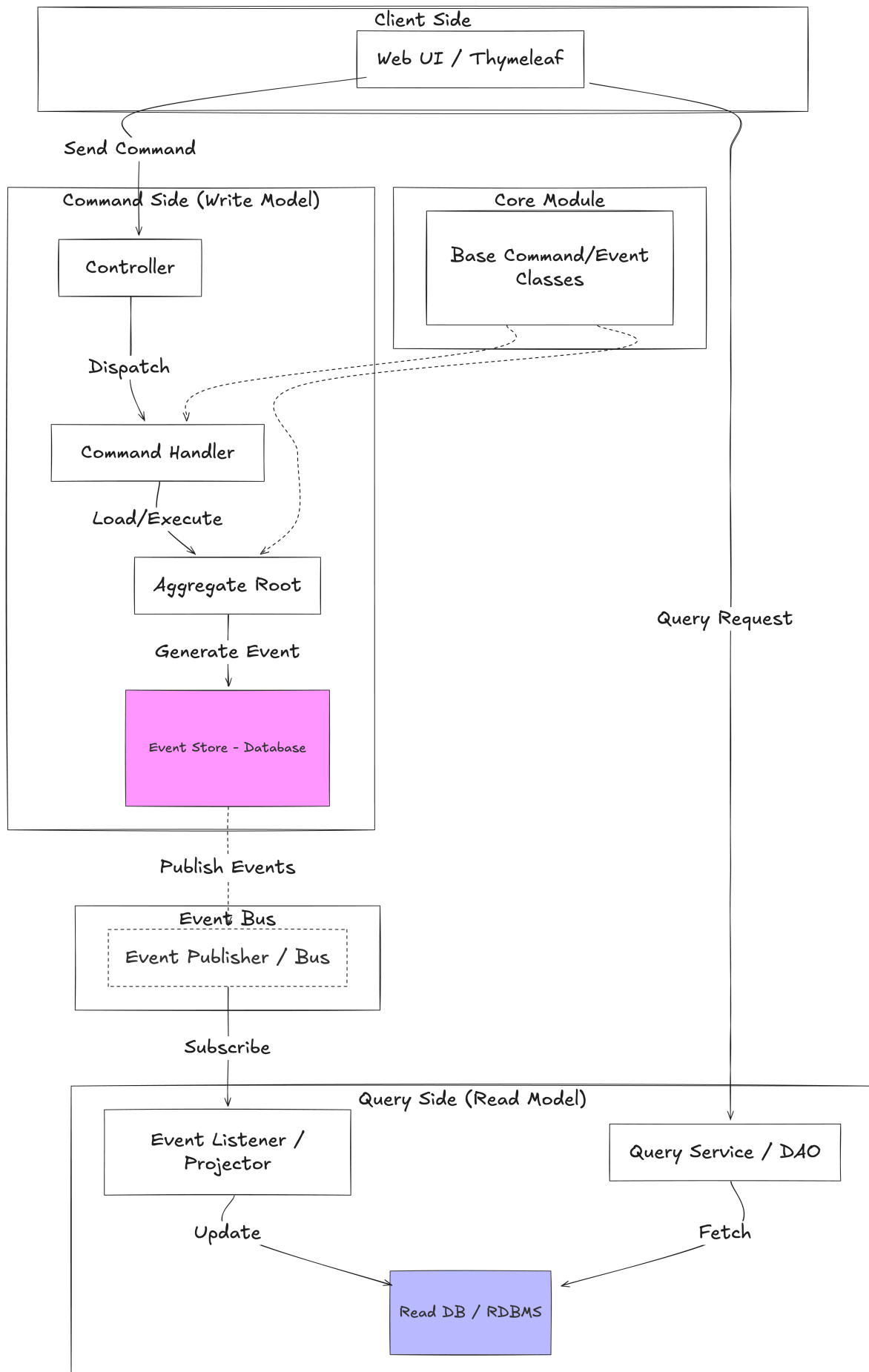
예를 들어, 100만개의 계좌 중 최근 일주일 안에 거래 정지된 계좌를 찾아야 한다고 가정해 보자. 결국 총 100만개의 계좌 상태를 모두 재생하고 거래 정지 상태인 계좌를 찾아야 한다. 객체의 재생 속도는 이벤트의 개수에 따른 차이가 있겠지만, 스냅샷을 적용했다 하더라도 전통적인 저장 방식보다 훨씬 더 많은 시간이 소요된다.

쓰기 작업은 성능보다는 정합성이 더 중요하기에 Snapshot과 이벤트를 통한 재구성 작업이 반드시 필요하다. 하지만 위의 예시를 보면, 조회는 정합성의 트레이드 오프를 감수하더라도 성능이 더 중요함을 알 수 있다.

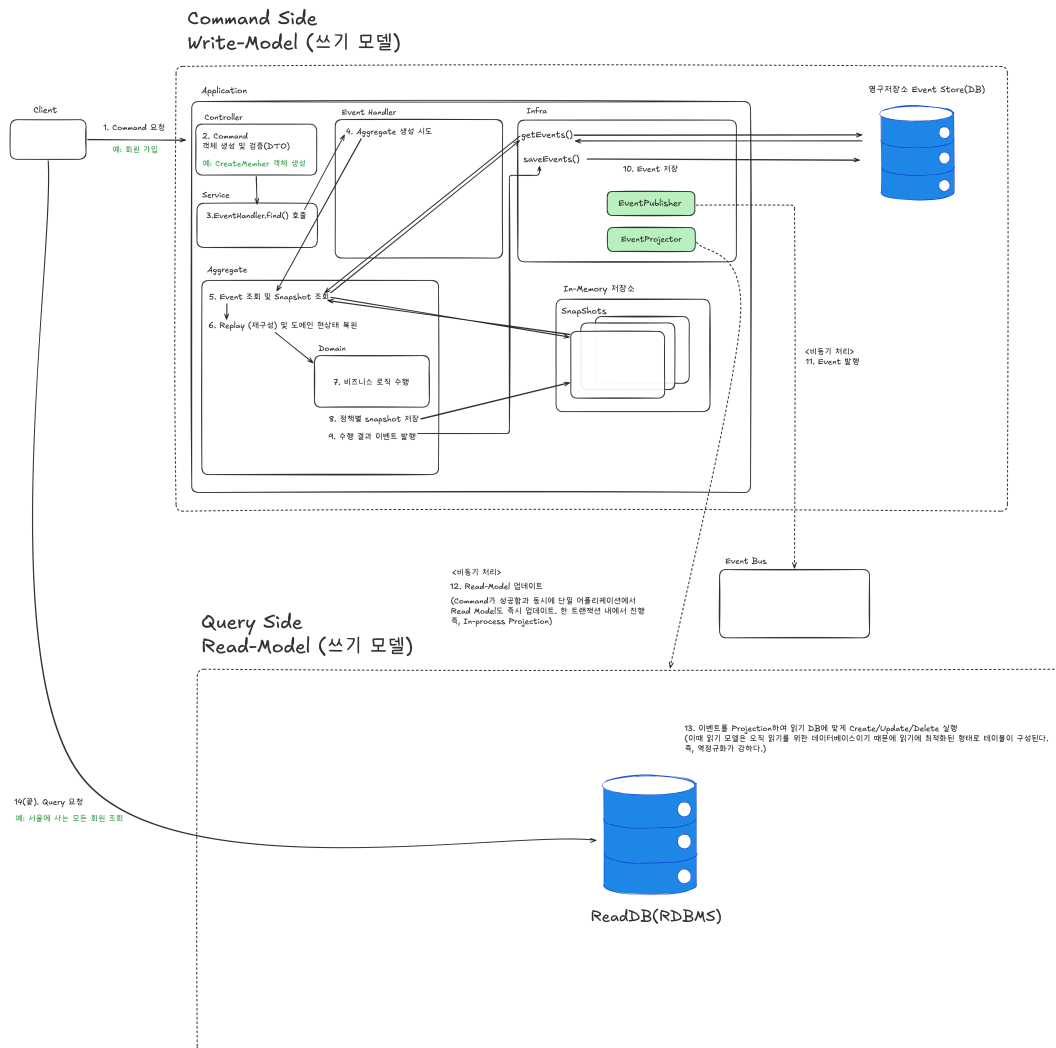
이때 CQRS 패턴을 도입하여, 별도의 읽기 모델을 생성하고 조회 전용으로 현상태 값을 별도로 저장한다면, 조회성능을 크게 개선할 수 있다. 특히 읽기 모델은 서비스 요구사항에 맞게 최적화 할 수 있기 때문에, 전통적인 저장방식보다 더 효율적인 읽기 성능을 제공할 수도 있다. 예를 들어, 중복된 데이터가 테이블 쌓이더라도, 이 테이블 설계가 조회에 유리하다면 이와 같이 역정규화된 테이블을 사용할 수도 있다.

3. Event Sourcing의 실행 매커니즘

이제 이론은 여기서 차치하고, 전체적인 시스템 설계를 살펴보며 이해하자.



3.1 Command(명령) 시퀀스



Command는 Aggregate를 이벤트로 복원한 뒤, 새로운 이벤트를 만들어 append하는 과정이다.

1. command 요청이 들어온다. 이때, command 객체를 만들고 이를 이벤트 핸들러로 보낸다. 여기서 command 객체는 우리가 일반적으로 입력받는 DTO로 해석하면 된다.
2. 대상 Aggregate를 재생(replay)한다. 여기서 Aggregate는 비즈니스 로직을 포함하는 도메인 객체를 의미한다. 즉 Aggregate 내부에서 비즈니스 로직 실행 이후의 이벤트를 발행하는 책임을 갖는다. 재생시에는 snapshot을 우선적으로 조회하고 이를 기반으로 재생한다.
3. 재생된 Aggregate 내부에서 비즈니스 로직이 실행된다. 이 실행 결과는 상태값 수정이 아닌 이벤트의 형식으로 메모리에 저장된다.

4. 이후, EventStore(영구 저장소)에 해당 event가 저장되며, 스냅샷 정책에 만족하는 경우 Snapshot은 인메모리에 저장된다. snapshot은 캐시 개념으로 이해하면 편하다.
5. 마지막으로 snapshot 비동기로 Event가 발행되고 projection을 통해 읽기 모델이 업데이트 된다.

중요한 점

- 도메인 로직 실행시 상태를 바로 바꾸지 않는다.
- 이벤트는 커밋 단위로 저장된다.
- 여기서 핵심은 CQRS 패턴을 도입해, 매번 재구성해야하는 읽기 단점의 성능을 보완했다는 점이다.

3.2 Query(조회)시퀀스

조회 메서드는 읽기 모델로 요청이 들어간다. 즉, 명령 요청시 projection을 통해 만들어진 읽기 모델을 읽기 때문에 별도의 replay가 진행되지 않는다. 즉, 읽기 성능을 포기하지 않을 수 있다.

해당 매커니즘으로 이벤트 소싱을 적용하는 경우 얻을 수 있는 장점

- 동시성 문제를 해결할 수 있다. 각 어그리게이트 재구성시점에 버전 정보를 저장한다. 이 버전 정보를 기반으로 저장 시점에 버전 체크를 진행하기에 동시성 문제를 해결 가능하다. (lost update 방지)
- 감사로그와 같이 이력이 남아야하는 경우 관리하기 편하다. 앞서 살펴봤듯이, 이력 테이블을 별도로 관리하지 않기에 이벤트만 재생해서 관리하기 용이하다.
- 조회와 읽기 모델의 분리로 읽기 성능도 올라간다.
- 디버깅 시점에 replay로 버그를 직접 재현 가능하다.

참고 래퍼런스

<https://mjspring.medium.com/%EC%9D%B4%EB%B2%A4%ED%8A%B8-%EC%86%8C%EC%8B%B1-event-sourcing-%EA%B0%9C%EB%85%90-50029f50f78c>

<https://www.youtube.com/watch?v=12EGxMB8SR8&list=PLdHtZnJh1KdZ6NDO9zc9hF4tONDLTSEUV&index=15>

<https://mslim8803.tistory.com/73>

